

Assignment-11.1

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
		Mr. Bingi Raju	
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week6 – Monday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 11.1 (Present assignment number)/ 24 (Total number of assignments)			
Q.No.	Question		Expected Time to

		complete
	Lab 11 – Data Structures with AI: Implementing Fundamental Structures Lab Objectives • Use AI to assist in designing and implementing fundamental data structures in Python. • Learn how to prompt AI for structure creation, optimization, and documentation. • Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables. • Enhance code quality with AI-generated comments and performance suggestions.	
1	Task Description #1 – Stack Implementation Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods. Sample Input Code: <pre>class Stack: pass</pre> Expected Output: • A functional stack implementation with all required methods and docstrings. Prompt: Generate a Python Stack class using a list with push, pop, peek, and is_empty methods, ensuring correct functionality and safe empty-stack handling. Code:	Week 6 - Monday

```

1  class Stack:
2      def __init__(self):
3          self.items = []
4
5      def push(self, item):
6          self.items.append(item)
7          print(f"Pushed: {item}")
8
9      def pop(self):
10         if self.is_empty():
11             print("Error: Cannot pop from an empty stack")
12             return None
13         return self.items.pop()
14
15     def peek(self):
16         if self.is_empty():
17             print("Error: Stack is empty")
18             return None
19         return self.items[-1]
20
21     def is_empty(self):
22         return len(self.items) == 0
23
24     def display(self):
25         if self.is_empty():
26             print("Stack is empty")
27         else:
28             print(f"Stack contents: {self.items}")
29
30
31     def main():
32         stack = Stack()
33
34         while True:
35             print("\n--- Stack Operations ---")
36             print("1. Push")
37             print("2. Pop")
38             print("3. Peek")
39             print("4. Is Empty")
40             print("5. Display Stack")
41             print("6. Exit")
42
43             choice = input("Enter your choice (1-6): ").strip()
44
45             if choice == '1':
46                 value = input("Enter value to push: ").strip()
47                 stack.push(value)
48
49             elif choice == '2':
50                 popped = stack.pop()
51                 if popped is not None:
52                     print(f"Popped: {popped}")
53
54             elif choice == '3':
55                 peeked = stack.peek()
56                 if peeked is not None:
57                     print(f"Top element: {peeked}")
58
59             elif choice == '4':
60                 if stack.is_empty():
61                     print("Stack is empty: True")
62                 else:
63                     print("Stack is empty: False")
64
65             elif choice == '5':
66                 stack.display()
67
68             elif choice == '6':
69                 print("Exiting...")
70                 break
71
72             else:
73                 print("Invalid choice. Please try again.")
74
75
76     if __name__ == "__main__":
77         main()
78

```



```
Stack contents: ['2', '3']
```

```
--- Stack Operations ---
```

```
1. Push
```

```
2. Pop
```

```
3. Peek
```

```
4. Is Empty
```

```
5. Display Stack
```

```
6. Exit
```

```
Enter your choice (1-6): 6
```

```
Exiting...
```

```
PS C:\Users\Naveen\Desktop\AI Coding> █
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Prompt:

Implement a FIFO-based Python Queue class using a list with enqueue, dequeue, peek, and size methods, where all operations dynamically take input from the user at runtime and handle empty queue cases safely.

Output:

```
1 class Queue:
2     def __init__(self):
3         self.items = []
4
5     def enqueue(self, item):
6         self.items.append(item)
7         print(f"Enqueued: {item}")
8
9     def dequeue(self):
10        if self.is_empty():
11            print("Queue is empty. Cannot dequeue.")
12            return None
13        item = self.items.pop(0)
14        print(f"Dequeued: {item}")
15        return item
16
17    def peek(self):
18        if self.is_empty():
19            print("Queue is empty. Cannot peek.")
20            return None
21        print(f"Front element: {self.items[0]}")
22        return self.items[0]
23
24    def size(self):
25        return len(self.items)
26
27    def is_empty(self):
28        return len(self.items) == 0
29
30    def display(self):
31        if self.is_empty():
32            print("Queue is empty.")
33        else:
34            print(f"Queue: {self.items}")
35
36
37 def main():
38     queue = Queue()
39
40     while True:
41         print("\n--- Queue Operations ---")
42         print("1. Enqueue")
43         print("2. Dequeue")
44         print("3. Peek")
45         print("4. Size")
46         print("5. Display Queue")
47         print("6. Exit")
48
49         choice = input("Enter your choice (1-6): ").strip()
50
51         if choice == '1':
52             try:
53                 item = input("Enter item to enqueue: ").strip()
54                 queue.enqueue(item)
55             except Exception as e:
56                 print(f"Error: {e}")
57
58         elif choice == '2':
59             queue.dequeue()
60
61         elif choice == '3':
62             queue.peek()
63
64         elif choice == '4':
65             print(f"Queue size: {queue.size()}")
66
67         elif choice == '5':
68             queue.display()
69
70         elif choice == '6':
71             print("Exiting...")
72             break
73
74         else:
75             print("Invalid choice. Please try again.")
76
77
78 if __name__ == "__main__":
79     main()
80
```

```
Queue: ['4', '5']

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Peek
4. Size
5. Display Queue
6. Exit
Enter your choice (1-6): 6
Exiting...
PS C:\Users\Naveen\Desktop\AI Coding> 
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
    pass
```

```
class LinkedList:
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Prompt:

Generate a Python implementation of a Singly Linked List using Node and LinkedList classes, with insert and display methods where node values are dynamically taken from the user at runtime, ensuring correct linked list behavior and clear method documentation.

Code:

Output:

```
1 class Node:
2     """
3     Represents a single node in the linked list.
4
5     Attributes:
6         data: The value stored in the node
7         next: Reference to the next node in the linked list
8     """
9     def __init__(self, data):
10         """
11         Initialize a node with the given data.
12
13         Args:
14             data: The value to store in the node
15         """
16         self.data = data
17         self.next = None
18
19
20 class LinkedList:
21     """
22     Implements a singly linked list with insertion and display functionality.
23
24     Attributes:
25         head: Reference to the first node in the linked list
26     """
27     def __init__(self):
28         """Initialize an empty linked list."""
29         self.head = None
30
31     def insert(self, data):
32         """
33         Insert a new node with the given data at the end of the linked list.
34
35         Args:
36             data: The value to insert into the linked list
37         """
38         new_node = Node(data)
39
40         # If the list is empty, the new node becomes the head
41         if self.head is None:
42             self.head = new_node
43             return
44
45         # Traverse to the last node
46         current = self.head
47         while current.next is not None:
48             current = current.next
49
50         # Insert the new node at the end
51         current.next = new_node
52
53     def display(self):
54         """
55         Display all nodes in the linked list in order.
56         Prints the linked list from head to tail.
57         """
58         if self.head is None:
59             print("Linked List is empty.")
60             return
61
62         print("Linked List: ", end="")
63         current = self.head
64         while current is not None:
65             print(current.data, end=" -> ")
66             current = current.next
67         print("None")
68
69
70 # Main program
71 if __name__ == "__main__":
72     linked_list = LinkedList()
73
74     print("Singly Linked List Implementation")
75     print("-" * 40)
76
77     # Take input from user
78     try:
79         n = int(input("Enter the number of nodes: "))
80
81         if n <= 0:
82             print("Please enter a positive number.")
83         else:
84             for i in range(n):
85                 value = input(f"Enter value for node {i + 1}: ")
86                 linked_list.insert(value)
87
88             # Display the linked list
89             print("\n")
90             linked_list.display()
91
92     except ValueError:
93         print("Invalid input. Please enter valid data.")
94
```



```
PS C:\Users\Havveen\Desktop\AI Coding> & C:\Users\Havveen\AppData\Local\Programs\Python\Python314\python.exe -c "c:/Users/Havveen/Desktop/AI Coding/11_02.py"
Singly Linked List Implementation
-----
Enter the number of nodes: 3
Enter value for node 1: 1
Enter value for node 2: 2
Enter value for node 3: 3

Linked List: 1 -> 2 -> 3 -> None
PS C:\Users\Havveen\Desktop\AI Coding>
```

Explanation:

This task implements a singly linked list using Node and LinkedList classes.

The program dynamically accepts user input to insert elements into the list.

The display method prints all nodes in sequence, maintaining proper links between them.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
    pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.

Prompt:

Generate a Python Binary Search Tree (BST) implementation using a BST class with recursive insert and in-order traversal methods.

The tree should dynamically accept input from the user at runtime for node insertion and correctly display elements using in-order traversal.

Output:

```
1 class Node:
2     def __init__(self, value):
3         self.value = value
4         self.left = None
5         self.right = None
6
7
8 class BST:
9     def __init__(self):
10         self.root = None
11
12     def insert(self, value):
13         if self.root is None:
14             self.root = Node(value)
15         else:
16             self._insert_recursive(self.root, value)
17
18     def _insert_recursive(self, node, value):
19         if value < node.value:
20             if node.left is None:
21                 node.left = Node(value)
22             else:
23                 self._insert_recursive(node.left, value)
24         elif value > node.value:
25             if node.right is None:
26                 node.right = Node(value)
27             else:
28                 self._insert_recursive(node.right, value)
29
30     def inorder_traversal(self):
31         result = []
32         self._inorder_recursive(self.root, result)
33         return result
34
35     def _inorder_recursive(self, node, result):
36         if node is not None:
37             self._inorder_recursive(node.left, result)
38             result.append(node.value)
39             self._inorder_recursive(node.right, result)
40
41     def display(self):
42         traversal = self.inorder_traversal()
43         if traversal:
44             print("In-order Traversal:", traversal)
45         else:
46             print("Tree is empty")
47
48
49 def main():
50     bst = BST()
51     print("Binary Search Tree (BST) Implementation")
52     print("=" * 40)
53
54     while True:
55         print("\nOptions:")
56         print("1. Insert a value")
57         print("2. Display in-order traversal")
58         print("3. Exit")
59
60         choice = input("Enter your choice (1/2/3): ").strip()
61
62         if choice == '1':
63             try:
64                 value = int(input("Enter the value to insert: "))
65                 bst.insert(value)
66                 print(f"Value {value} inserted successfully!")
67             except ValueError:
68                 print("Invalid input! Please enter an integer.")
69
70         elif choice == '2':
71             bst.display()
72
73         elif choice == '3':
74             print("Exiting the program. Goodbye!")
75             break
76
77         else:
78             print("Invalid choice! Please enter 1, 2, or 3.")
79
80
81 if __name__ == "__main__":
82     main()
83
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
2. Display in-order traversal
3. Exit
Enter your choice (1/2/3): 2
In-order Traversal: [2, 3]
```

```
Options:
1. Insert a value
2. Display in-order traversal
3. Exit
Enter your choice (1/2/3): 3
Exiting the program. Goodbye!
PS C:\Users\Naveen\Desktop\AI Coding> █
```

Explanation:

This task implements a Binary Search Tree (BST) where nodes are inserted recursively based on BST rules.

User inputs are taken dynamically to build the tree at runtime.

An in-order traversal is used to display the elements in sorted order.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
    pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Prompt:

Generate a Python Hash Table implementation using a HashTable class with insert, search, and delete methods, where all keys and values are dynamically taken from the user at runtime, and collision handling is done using chaining.

Code:

Output:

```
1 class HashTable:
2     def __init__(self, size=10):
3         """Initialize hash table with given size using chaining for collision handling"""
4         self.size = size
5         self.table = [[] for _ in range(size)]
6
7     def _hash(self, key):
8         """Generate hash value for a given key"""
9         return hash(key) % self.size
10
11    def insert(self, key, value):
12        """Insert key-value pair into hash table"""
13        index = self._hash(key)
14
15        # Check if key already exists and update it
16        for i, (k, v) in enumerate(self.table[index]):
17            if k == key:
18                self.table[index][i] = (key, value)
19                print(f"Updated: {key} -> {value}")
20                return
21
22        # If key doesn't exist, append new key-value pair
23        self.table[index].append((key, value))
24        print(f"Inserted: {key} -> {value}")
25
26    def search(self, key):
27        """Search for a key in hash table"""
28        index = self._hash(key)
29
30        for k, v in self.table[index]:
31            if k == key:
32                print(f"Found: {key} -> {v}")
33                return v
34
35        print(f"Key '{key}' not found")
36        return None
37
38    def delete(self, key):
39        """Delete a key from hash table"""
40        index = self._hash(key)
41
42        for i, (k, v) in enumerate(self.table[index]):
43            if k == key:
44                self.table[index].pop(i)
45                print(f"Deleted: {key}")
46                return True
47
48        print(f"Key '{key}' not found for deletion")
49        return False
50
51    def display(self):
52        """Display all key-value pairs in hash table"""
53        print("\n--- Hash Table Contents ---")
54        for i, chain in enumerate(self.table):
55            print(f"Index {i}: {chain}")
56        print("-----\n")
57
58    # Main program
59    if __name__ == "__main__":
60        size = int(input("Enter hash table size: "))
61        ht = HashTable(size)
62
63        while True:
64            print("\n1. Insert")
65            print("2. Search")
66            print("3. Delete")
67            print("4. Display")
68            print("5. Exit")
69
70            choice = input("Enter your choice (1-5): ").strip()
71
72            if choice == '1':
73                key = input("Enter key: ").strip()
74                value = input("Enter value: ").strip()
75                ht.insert(key, value)
76
77            elif choice == '2':
78                key = input("Enter key to search: ").strip()
79                ht.search(key)
80
81            elif choice == '3':
82                key = input("Enter key to delete: ").strip()
83                ht.delete(key)
84
85            elif choice == '4':
86                ht.display()
87
88            elif choice == '5':
89                print("Exiting program...")
90                break
91
92            else:
93                print("Invalid choice! Please try again.")
94
95
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

--- Hash Table Contents ---
Index 0: []
Index 1: []
Index 2: [('2', '6')]
-----

1. Insert
2. Search
3. Delete
4. Display
5. Exit
```

Explanation:

This task implements a hash table that stores key–value pairs using chaining to handle collisions. All insert, search, and delete operations take input dynamically from the user. The implementation ensures efficient data access while handling collisions correctly.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
    pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Prompt:

Generate a Python Graph implementation using an adjacency list in a Graph class.

Implement methods to add vertices, add edges, and display graph connections, where all vertices and edges are dynamically taken from the user at runtime, ensuring correct representation of the graph

Code:

Output:

```
1 class Graph:
2     def __init__(self):
3         self.graph = {}
4
5     def add_vertex(self, vertex):
6         if vertex not in self.graph:
7             self.graph[vertex] = []
8             print(f"Vertex {vertex} added successfully.")
9         else:
10            print(f"Vertex {vertex} already exists.")
11
12    def add_edge(self, u, v):
13        if u not in self.graph:
14            print(f"Vertex {u} does not exist. Add it first.")
15            return
16        if v not in self.graph:
17            print(f"Vertex {v} does not exist. Add it first.")
18            return
19
20        if v not in self.graph[u]:
21            self.graph[u].append(v)
22            print(f"Edge from {u} to {v} added successfully.")
23        else:
24            print(f"Edge from {u} to {v} already exists.")
25
26    def display_graph(self):
27        if not self.graph:
28            print("Graph is empty.")
29            return
30
31        print("\nGraph Connections:")
32        for vertex in self.graph:
33            if self.graph[vertex]:
34                print(f"{vertex} -> {' '.join(map(str, self.graph[vertex]))}")
35            else:
36                print(f"{vertex} -> (no connections)")
37
38
39    def main():
40        g = Graph()
41
42        while True:
43            print("\n=== Graph Operations ===")
44            print("1. Add Vertex")
45            print("2. Add Edge")
46            print("3. Display Graph")
47            print("4. Exit")
48
49            choice = input("Enter your choice (1/2/3/4): ").strip()
50
51            if choice == '1':
52                vertex = input("Enter vertex to add: ").strip()
53                g.add_vertex(vertex)
54
55            elif choice == '2':
56                u = input("Enter source vertex: ").strip()
57                v = input("Enter destination vertex: ").strip()
58                g.add_edge(u, v)
59
60            elif choice == '3':
61                g.display_graph()
62
63            elif choice == '4':
64                print("Exiting...")
65                break
66
67            else:
68                print("Invalid choice. Please try again.")
69
70
71    if __name__ == "__main__":
72        main()
73
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Graph Connections:
2 -> (no connections)
3 -> (no connections)

=== Graph Operations ===
1. Add Vertex
2. Add Edge
3. Display Graph
4. Exit
Enter your choice (1/2/3/4): 4
Exiting...
PS C:\Users\Naveen\Desktop\AI Coding> █
```

Explanation:

This task implements a graph using an adjacency list representation. Vertices and edges are added dynamically based on user input. The display method shows all connections between vertices clearly.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue:
    pass
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Prompt:

Generate a Python Priority Queue implementation using the heapq module in a PriorityQueue class.

Implement enqueue (with priority), dequeue (highest priority), and display methods, where all elements and priorities are dynamically taken from the user at runtime, ensuring correct priority-based ordering.

Code:

Output:

```
1 import heapq
2
3 class PriorityQueue:
4     def __init__(self):
5         self.heap = []
6
7     def enqueue(self, element, priority):
8         """Add an element with a given priority (lower number = higher priority)"""
9         heapq.heappush(self.heap, (priority, element))
10        print(f"Enqueued: {element} with priority {priority}")
11
12    def dequeue(self):
13        """Remove and return the element with highest priority"""
14        if not self.heap:
15            print("Priority Queue is empty!")
16            return None
17        priority, element = heapq.heappop(self.heap)
18        print(f"Dequeued: {element} with priority {priority}")
19        return element
20
21    def display(self):
22        """Display all elements in the priority queue"""
23        if not self.heap:
24            print("Priority Queue is empty!")
25            return
26        print("\nCurrent Priority Queue:")
27        sorted_heap = sorted(self.heap)
28        for priority, element in sorted_heap:
29            print(f"Element: {element}, Priority: {priority}")
30        print()
31
32    def main():
33        pq = PriorityQueue()
34
35        while True:
36            print("\n--- Priority Queue Operations ---")
37            print("1. Enqueue")
38            print("2. Dequeue")
39            print("3. Display")
40            print("4. Exit")
41
42            choice = input("Enter your choice (1-4): ").strip()
43
44            if choice == '1':
45                element = input("Enter element: ").strip()
46                try:
47                    priority = int(input("Enter priority (lower number = higher priority): ").strip())
48                    pq.enqueue(element, priority)
49                except ValueError:
50                    print("Invalid priority! Please enter an integer.")
51
52            elif choice == '2':
53                pq.dequeue()
54
55            elif choice == '3':
56                pq.display()
57
58            elif choice == '4':
59                print("Exiting...")
60                break
61
62            else:
63                print("Invalid choice! Please try again.")
64
65    if __name__ == "__main__":
66        main()
67
```

Output:


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Current Priority Queue:
  Element: 3, Priority: 2
  Element: 2, Priority: 5

--- Priority Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice (1-4): 4
Exiting...
```

Explanation:

This task implements a priority queue using Python's `heapq` module. Elements along with their priorities are taken dynamically from the user. The queue always removes the element with the highest priority first.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Sample Input Code:

```
class DequeDS:
```

```
    pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

Prompt:

Generate a Python double-ended queue (Deque) implementation using `collections.deque` in a `DequeDS` class.

Implement methods to insert and remove elements from both the front and rear, where all inputs are dynamically taken from the user at runtime, and include clear docstrings for each method.

Code:

```

1 from collections import deque
2
3
4 class DequeDS:
5     """
6     A double-ended queue (Deque) implementation using collections.deque.
7     Supports insertion and removal from both front and rear ends.
8     """
9
10    def __init__(self):
11        """Initialize an empty deque."""
12        self.deque = deque()
13
14    def insert_front(self, value):
15        """
16        Insert an element at the front of the deque.
17
18        Args:
19            value: The element to insert at the front.
20        """
21        self.deque.appendleft(value)
22        print(f"Inserted {value} at the front.")
23
24    def insert_rear(self, value):
25        """
26        Insert an element at the rear of the deque.
27
28        Args:
29            value: The element to insert at the rear.
30        """
31        self.deque.append(value)
32        print(f"Inserted {value} at the rear.")
33
34    def remove_front(self):
35        """
36        Remove and return an element from the front of the deque.
37
38        Returns:
39            The element removed from the front, or None if deque is empty.
40        """
41        if self.is_empty():
42            print("Deque is empty. Cannot remove from front.")
43            return None
44        value = self.deque.popleft()
45        print(f"Removed {value} from the front.")
46        return value
47
48    def remove_rear(self):
49        """
50        Remove and return an element from the rear of the deque.
51
52        Returns:
53            The element removed from the rear, or None if deque is empty.
54        """
55        if self.is_empty():
56            print("Deque is empty. Cannot remove from rear.")
57            return None
58        value = self.deque.pop()
59        print(f"Removed {value} from the rear.")
60        return value
61
62    def is_empty(self):
63        """
64        Check if the deque is empty.
65
66        Returns:
67            True if deque is empty, False otherwise.
68        """
69        return len(self.deque) == 0
70
71    def display(self):
72        """Display all elements in the deque."""
73        if self.is_empty():
74            print("Deque is empty.")
75        else:
76            print(f"Deque contents: {list(self.deque)}")
77
78
79 def main():
80     """Main function to interact with the DequeDS."""
81     deque_ds = DequeDS()
82
83     while True:
84         print("\n-- Deque Operations --")
85         print("1. Insert at front")
86         print("2. Insert at rear")
87         print("3. Remove from front")
88         print("4. Remove from rear")
89         print("5. Display deque")
90         print("6. Exit")
91
92         choice = input("Enter your choice (1-6): ").strip()
93
94         if choice == '1':
95             value = input("Enter value to insert at front: ").strip()
96             deque_ds.insert_front(value)
97
98         elif choice == '2':
99             value = input("Enter value to insert at rear: ").strip()
100            deque_ds.insert_rear(value)
101
102        elif choice == '3':
103            deque_ds.remove_front()
104
105        elif choice == '4':
106            deque_ds.remove_rear()
107
108        elif choice == '5':
109            deque_ds.display()
110
111        elif choice == '6':
112            print("Exiting...")
113            break
114
115        else:
116            print("Invalid choice. Please try again.")
117
118
119 if __name__ == "__main__":
120     main()
121

```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Deque contents: ['6', '3', '4']

--- Deque Operations ---
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Display deque
6. Exit
Enter your choice (1-6): 6
Exiting...
PS C:\Users\Naveen\Desktop\AI Coding> 
```

Explanation:

This task implements a double-ended queue (deque) using Python's collections.deque.

Elements are inserted and removed from both ends based on dynamic user input.

The implementation allows efficient operations at both the front and rear of the queue.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below: ○ Stack ○ Queue
 - Priority Queue ○
 - Linked List ○ Binary
 - Search Tree (BST) ○ Graph ○
 - Hash Table

- Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

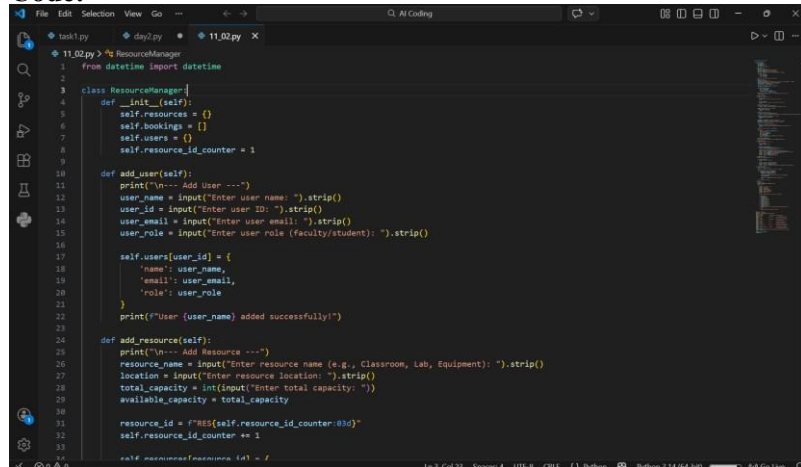
Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

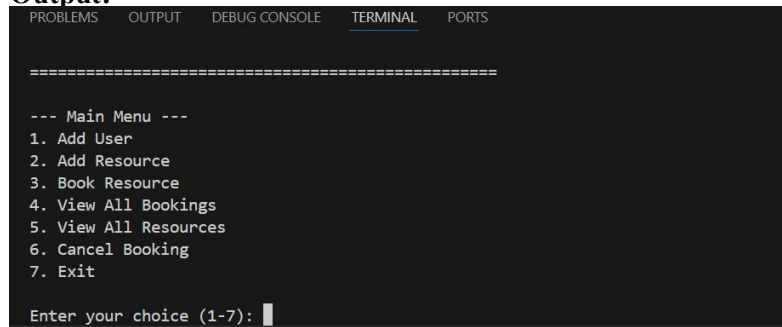
Select the most suitable data structure for each feature of a Campus Resource Management System, justify each choice in 2–3 sentences in a table, and implement one selected feature as a Python program that dynamically takes user input with comments and docstrings.

Code:



```
1 from datetime import datetime
2
3 class ResourceManager:
4     def __init__(self):
5         self.resources = {}
6         self.bookings = []
7         self.users = {}
8         self.resource_id_counter = 1
9
10    def add_user(self):
11        print("\n--- Add User ---")
12        user_name = input("Enter user name: ").strip()
13        user_id = input("Enter user ID: ").strip()
14        user_email = input("Enter user email: ").strip()
15        user_role = input("Enter user role (faculty/student): ").strip()
16
17        self.users[user_id] = {
18            'name': user_name,
19            'email': user_email,
20            'role': user_role
21        }
22        print(f"User {user_name} added successfully!")
23
24    def add_resource(self):
25        print("\n--- Add Resource ---")
26        resource_name = input("Enter resource name (e.g., Classroom, Lab, Equipment): ").strip()
27        location = input("Enter resource location: ").strip()
28        total_capacity = int(input("Enter total capacity: "))
29        available_capacity = total_capacity
30
31        resource_id = f"RES{self.resource_id_counter:03d}"
32        self.resource_id_counter += 1
33
34        self.resources[resource_id] = {
```

Output:



```
=====

--- Main Menu ---
1. Add User
2. Add Resource
3. Book Resource
4. View All Bookings
5. View All Resources
6. Cancel Booking
7. Exit

Enter your choice (1-7):
```

Explanation:

This task analyzes real-time campus system features and maps each one to the most suitable data structure. Each choice is justified based on efficiency and use-case requirements. One feature is implemented as a dynamic Python program using user input to demonstrate practical application.

	scription #10: Smart E-Commerce Platform – Data Structure Challenge An e-commerce company wants to build a Smart Online Shopping System with: 1. Shopping Cart Management – Add and remove products dynamically. 2. Order Processing System – Orders processed in the order they are placed. 3. Top-Selling Products Tracker – Products ranked by sales count. 4. Product Search Engine – Fast lookup of products using product ID. 5. Delivery Route Planning – Connect warehouses and delivery locations. Student Task: • For each feature, select the most appropriate data structure from the list below: ○ Stack ○ Queue ○ Priority Queue ○ Linked List ○ Binary Search Tree (BST) ○ Graph ○ Hash Table ○ Deque • Justify your choice in 2–3 sentences per feature. • Implement one selected feature as a working Python program with AI-assisted code generation. Expected Output: • A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings. Prompt: Map each e-commerce feature to the most suitable data structure with 2–3 line justifications in a table, and implement one chosen feature as a Python program that dynamically takes user input with comments and docstrings. Code:	
--	--	--

```

11_02.py > ShoppingCart > @update_quantity
# Update Reviews : def - AVULAS Updates Reviews, Test Membership Checking.
8
"wishlist": "Set - No duplicates, fast add/remove operations.",
9
"Product Categories": "Tree/Graph - Hierarchical relationships, category-subcategory mapping.",
10
"Search Index": "Hash Table - O(1) search by product name or keyword.",
11
"Order History": "Stack/List - Recent orders accessed first, maintains chronological order."
12
}
13
print("E-Commerce Features & Data Structures Mapping:\n")
14
for feature, structure in e_commerce_features.items():
15
    print(f"{feature}: {structure}\n")
16
17
print("\n" + "="*60)
18
print("SHOPPING CART IMPLEMENTATION")
19
print("="*60 + "\n")
20
21
class ShoppingCart:
22
    def __init__(self, user_id):
23
        self.user_id = user_id
24
        self.cart = []
25
26
    def add_item(self, product_id, product_name, price, quantity):
27
        for item in self.cart:
28
            if item['product_id'] == product_id:
29
                item['quantity'] += quantity
30
                print(f"Updated {product_name} quantity to {item['quantity']}")
31
            return
32
        self.cart.append({
33
            'product_id': product_id,
34
            'product_name': product_name,
35
            'price': price,
36
            'quantity': quantity
37
        })
38

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter User ID: 23

1. Add Item  2. Remove Item  3. Update Qty  4. View Cart  5. Checkout  6. Exit
Select option: 1
Enter Product ID: 234
Enter Product Name: book
Enter Price: 200
Enter Quantity: 3
Added book (x3) to cart

1. Add Item  2. Remove Item  3. Update Qty  4. View Cart  5. Checkout  6. Exit
Select option: 

```

Explanation:

This task selects appropriate data structures for different features of a smart e-commerce system.

Each choice is justified based on efficiency and real-time requirements. One feature is implemented as a dynamic Python program to demonstrate practical usage.